

The Joy of Typesetting Cryptography

How to make it look like *The Joy of Cryptography*

Mike Rosulek, <https://joyofcryptography.com>

April 17, 2024

This is a self-contained example of the formatting used in *The Joy of Cryptography*. Please consult the source code of this document.

A few convenient macros

<code>\bit{0}</code>	0
<code>\bits</code>	$\{0, 1\}$
<code>\indist</code>	$\cong$
<code>\gets</code>	$\leftarrow$
<code>\lib{test}</code>	$\mathcal{L}_{\text{test}}$
<code>\hl{highlighted} text</code>	highlighted text
math mode: <code>\$X \hl{\gets \bits^{\secpars}}\$</code>	math mode: $X \leftarrow \{0, 1\}^\lambda$

“codebox” syntax:

The `\begin{pipetex}` environment passes its body to the `codebox2tex.pl` script, and renders the output as TeX. See <https://github.com/rosulek/pipetex.sty/>.

You need to compile with the `-shell-escape` option to call external scripts!

Here is a brief overview of the `codebox2tex.pl` domain-specific language:

- Inside code ... end you can write multiple lines vertically. Outside of code, linebreaks don’t matter.

code	
<code>K \gets \bits^{\secpars}</code>	
<code>Y := F(K,X)</code>	$K \leftarrow \{0, 1\}^\lambda$
<code>Z := F(K,Y)</code>	$Y := F(K, X) \quad A := 1 \quad B := 2$
end	$Z := F(K, Y)$
<code>A := 1</code>	
<code>B := 2</code>	

- `\lib ... end` is similar, but draws a box. `\lib` takes an optional argument. You can have more than one `\lib/code` box (often separated by `\link`, `\indist`, etc):

```

lib
  K \gets \bits^\secp
  Y := F(K,X)
  Z := F(K,Y)
end
\link
lib \lib{test}
  R \gets \bits^\secp
  U := R \oplus Z
end
\indist
\cdots

```

$$\boxed{\begin{array}{l} K \leftarrow \{0,1\}^\lambda \\ Y := F(K,X) \\ Z := F(K,Y) \end{array}} \diamond \boxed{\begin{array}{c} \mathcal{L}_{\text{test}} \\ R \leftarrow \{0,1\}^\lambda \\ U := R \oplus Z \end{array}} \cong \dots$$

- Lines that contain no \$ are assumed to be entirely math (exception: lines like “else:”). Lines that start with // are rendered as comments.

```

lib
  // math:
  K \gets \bits^\secp

  // mixture of math and non-math:
  add $K$ to the set $\mathcal{K}$
end

```

```

// math:
K ← {0,1}^λ

// mixture of math and non-math:
add K to the set K

```

- Four spaces for indentation:

```

lib
  K \gets \bits^\secp

  if $K == \bit0^\secp$:
    for $i=1$ to $n$:
      print $\bit{oops}$
    end
end

```

```

K ← {0,1}^λ

if K == 0^λ:
  for i = 1 to n:
    print oops

```

- Start a subroutine with proc. If the subroutine name contains only [a-zA-Z\.] then it is rendered nicely (with \subname), otherwise assumed to be math:

```

lib
  proc otp.enc(M)
    K \gets \bits^n
    return $K \oplus M$

  proc \mathbb{H}^+(A)
    if $H[A]$ undefined:
      H[A] \gets \bits^n
    return $H[A]$
end

```

```

OTP.ENC(X):
  K ← {0,1}^n
  return K ⊕ X

H+(A):
  if H[A] undefined:
    H[A] ← {0,1}^n
  return H[A]

```

Sample (from homework)

Prove that the following encryption scheme has CPA security (real vs random) if F is a secure PRF (PRP):

$$\begin{array}{lcl}
 \mathcal{K} = \{0, 1\}^\lambda & & \text{Enc}(K, M): \\
 \mathcal{M} = \{0, 1\}^\lambda & & \quad R \leftarrow \{0, 1\}^\lambda \\
 C = \{0, 1\}^{2\lambda} & & \quad X := F(K, R) \\
 & & \quad Y := F(K, M) \\
 & & \quad Z := F(K, R \oplus Y) \\
 & & \quad \text{return } X \| Z
 \end{array}$$

In other words, prove that:

$$\begin{array}{ccccc}
 \text{if} & \boxed{\begin{array}{l} K \leftarrow \{0, 1\}^\lambda \\ \text{PRF.QUERY}(X): \\ \quad \text{return } F(K, X) \end{array}} & \cong & \boxed{\begin{array}{l} \text{PRF.QUERY}(X): \\ \quad \text{if } L[X] \text{ undefined:} \\ \quad \quad L[X] \leftarrow \{0, 1\}^\lambda \\ \quad \text{return } L[X] \end{array}} & \text{then} \\
 & & & & \boxed{\begin{array}{l} K \leftarrow \{0, 1\}^\lambda \\ \text{CPA.ENC}(M): \\ \quad // \text{Enc}(K, M): \\ \quad \quad R \leftarrow \{0, 1\}^\lambda \\ \quad \quad X := F(K, R) \\ \quad \quad Y := F(K, M) \\ \quad \quad Z := F(K, R \oplus Y) \\ \quad \quad \text{return } X \| Z \end{array}} \\
 & & & & \cong \\
 & & & & \boxed{\begin{array}{l} \text{CPA.ENC}(M): \\ \quad X \leftarrow \{0, 1\}^\lambda \\ \quad Z \leftarrow \{0, 1\}^\lambda \\ \quad \text{return } X \| Z \end{array}}
 \end{array}$$